

《面向对象程序设计》之——

异 常

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

讲授内容

- 异常处理的好处
- 异常的抛出和传播
- 异常的捕获和处理

异常处理的意义 (1/4)

- 程序执行中需要处理异常
 - 动态分配空间时可能不会成功
 - 打开文件可能会失败
 - 除法运算时分母可能为0
 - 整数相乘可能溢出
 - 指针可能越界
 -
- 异常处理可以增加系统的鲁棒性

异常处理的意义 (2/4)

■ 异常处理方法一

- 使用选择语句 (`if...else...`)
- 判断异常情况，即时处理
- 正常程序流程和异常处理语句混在一起
- 程序员往往无法专注于正常流程编程

异常处理的意义 (3/4)

■ 异常处理方法二

- 使用C++异常处理机制
- 判断异常情况，发现异常后抛出异常
- 正常程序流程和异常处理模块分开
- 程序员可以专注于正常流程编程，异常处理模块稍后编写

异常处理的意义 (4/4)

■ C++异常处理机制

- 程序在产生错误后抛出异常
- 异常处理模块捕获并处理异常
- 异常处理机制一般无法使程序恢复正常执行
- 可以为程序提供有序的整理操作（如析构对象）

异常处理基础 (1/2)

- 关键字 `try`: 出错时产生异常的代码放在 `try` 块中
- 关键字 `throw`: `throw` 语句可以抛出任意类型的异常, 包括自定义类型
- 关键字 `catch`: `catch` 块 (异常处理器) 捕捉和处理异常

异常处理基础 (2/2)

- 一个异常处理器一般只捕捉一种类型的异常
- `try`块抛出异常后，程序控制离开`try`块
- 抛出异常后，程序在`try`块后面的`catch`块中逐个搜索合适的异常处理器
- 如果`try`块没有异常抛出，则程序跳过所有`catch`块
- 抛出异常之后，程序控制无法返回到抛出点
- `try`块可以直接或间接抛出异常

例子1: 除数为零的异常处理 (1/3)

```
#include <iostream>
#include <string.h>
//定义异常类MyException
class MyException
{public:
    MyException(char *str)
    {   msg = str;   }
    char * show()
    {   return msg; }
private:
    char *msg;
};
```

例子1: 除数为零的异常处理 (2/3)

//定义除法函数division, 除数为0时抛出异常。

```
double division(int dividend, int divisor)
{   if (divisor == 0)
    //抛出异常对象
    throw MyException(
        "error: divided by zero!");
    return (double)dividend/divisor;
}
```

例子1: 除数为零的异常处理 (3/3)

```
main()
{int a, b;
 double result;
 cout<<"Enter two integers (EOF:end) : ";
 while (cin>>a>>b) //
 {try
  {result = division(a,b);
   cout<<a<<" / " <<b<<" = "
    <<result<<endl;
  }
 catch (MyException e)
  {cout<<e.show()<<endl;}
 cout<<endl;
 cout<<"Enter two integers (EOF:end) : ";
 } return 0; }
```

程序运行结果



Enter two integers (EOF:end) :12 7

$12 / 7 = 1.71429$

Enter two integers (EOF:end) :2 0

error: divided by zero!

Enter two integers (EOF:end) :34 5

$34 / 5 = 6.8$

Enter two integers (EOF:end) :

步骤分析

- 将可能异常的放在try语句中;
- 若try内无异常, 则正常执行, catch无效;
- 若有异常, throw语句抛出异常, 跳出本函数, 转到上一级的函数;
- 该异常提供给try-catch结构, 寻找匹配的catch子句。当前抛出的是MyException类, 匹配上, 执行当前catch子语句。

步骤分析：一些细节

- 被检测部分必须放在try中，否则无效；
- try和catch间不能有其他语句，必须紧跟；
- 花括号不能省略（代表一整个语句块）；
- 一个try可以搭配多个catch；
- catch后的圆括号中可以只写异常信息的类型名，或者(...)（表示任何异常）
- 无匹配，调用系统的terminate

异常的抛出和传播 (1/3)

- 关键字 `throw` 可以带任何类型的操作数（任意表达式），包括自定义类型（如异常对象）
- 异常抛出后，最近匹配的异常处理器捕获该异常
- 如果没有匹配的异常处理器，则系统调用 `terminate` 函数，`terminate` 函数默认调用 `abort` 函数终止程序的执行

std::terminate

Defined in header `<exception>`

```
void terminate();
```

 (until C++11)

```
[[noreturn]] void terminate() noexcept;
```

 (since C++11)

`std::terminate()` is called by the C++ runtime when the program cannot continue for any of the following reasons:

- 1) An [exception is thrown](#) and not caught (it is implementation-defined whether any stack unwinding is done in this case).
- 2) A function directly invoked by the exception handling mechanism while handling an exception that has not yet been caught exits via an exception (e.g. a destructor of some local object, or a copy constructor constructing a catch-clause parameter).
- 3) The constructor or the destructor of a static [or thread-local](#) (since C++11) object throws an exception.
- 4) A function registered with `std::atexit` or `std::at_quick_exit` (since C++11) throws an exception.
- 5) A [dynamic exception specification](#) is violated and the default handler for `std::unexpected` is executed.
- 6) A non-default handler for `std::unexpected` throws an exception that violates the previously violated [dynamic exception specification](#), if the specification does not include `std::bad_exception`. (until C++17)
- 7) A [noexcept specification](#) is violated (it is implementation-defined whether any stack unwinding is done in this case).
- 8) `std::nested_exception::rethrow_nested` is called for an object that isn't holding a captured exception.
- 9) An exception is thrown from the initial function of `std::thread`. (since C++11)
- 10) A joinable `std::thread` is destroyed or assigned to.

例子2: 抛出各类异常, 并捕捉它们 (1/3)



```
#include <iostream>
main()
{ int a, myint;
  float myfloat;
  double mydouble;
  cout<<"Enter a integer (EOF to end) : ";
  while (cin>>a) //抛出不同类型的异常
  { try {
    switch(a % 3) {
    case 0:
      //输入整数为3的倍数时抛出整型异常
      myint = a;
      throw myint;
      break;
```

例子2: 抛出各类异常, 并捕捉它们 (2/3)



```
case 1: //抛出float类型异常
    myfloat = (float)a;
    throw myfloat;
    break;
```

```
case 2: //抛出double类型异常
    mydouble = a;
    throw mydouble;
    break;
```

```
default:
    break;
}
```

```
}
```

例子2: 抛出各类异常, 并捕捉它们 (3/3)



```
    catch (int e) //捕获整型异常
    {cout<<"Integer Exception: "<<e<<endl;
    }
    catch (float e) //捕获浮点类型异常
    {cout<<"Float Exception: "<<e<<endl;
    }
    catch (double e) //捕获双精度类型异常
    {cout<<"Double Exception: "<<e<<endl;
    }
    cout<<endl;
    cout<<"Enter a integer (EOF to end) : ";
}
return 0;
}
```

程序运行结果

```
Enter a integer (EOF to end) :10  
Float Exception: 10
```

```
Enter a integer (EOF to end) :11  
Double Exception: 11
```

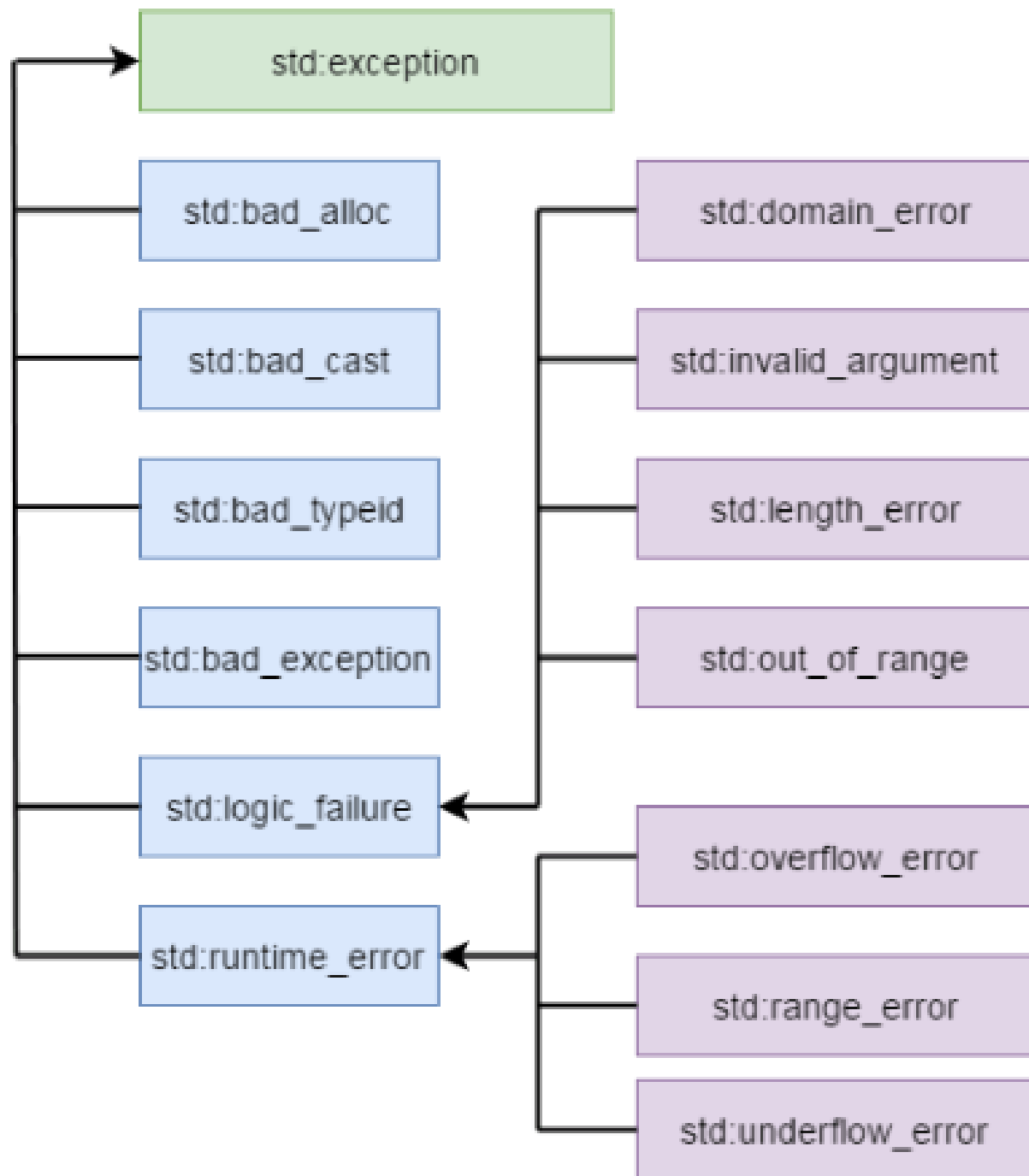
```
Enter a integer (EOF to end) :12  
Integer Exception: 12
```

```
Enter a integer (EOF to end) :13  
Float Exception: 13
```

```
Enter a integer (EOF to end) :
```

异常的抛出和传播 (2/3)

- 在try块中直接或间接抛出的异常，由其后匹配的catch块捕捉
- 在try块外面抛出的异常将不会被捕捉到，系统会调用terminate函数终止程序的运行
- 发生异常后跳出抛出异常的程序块，并且无法再返回到抛出点
- 异常可以在try块中显式抛出，也可以在其调用的函数中抛出



异常的抛出和传播 (3/3)

课本例8.2, 不错的例子!

- **try块可以嵌套**
- **内层try块抛出异常的传播顺序**
 - 先在内层try块后面的catch块中寻找合适的异常处理器
 - 找到则进行处理, 异常不再往外传播
 - 如果找不到, 则将该异常向外传播, 到外层try块后面的catch块中继续寻找
 - 如果异常传播到最外层的try块仍然找不到, 则程序调用terminate函数

```
#include <iostream>
#include<cmath>
using namespace std;
int main()
{
    void f1();
    try
    {
        f1();
    }
    catch (double)
    {
        cout << "ERROR0" << endl;

    }
    cout << "end0" << endl;
    return 0;
}
```

课本例8.2, 不错的例子!


```
void f1()
{
    void f2();
    try
    {
        f2();
    }
    catch (char)
    {
        cout << "ERROR1";
    }
    cout << "end1" << endl;
}
void f2()
{
    void f3();
    try
    {
        f3();
    }
    catch (int)
    {
        cout << "ERROR2";
    }
    cout << "end2" << endl;}
```

```
void f3()  
{  
    double a = 0;  
    try  
    {  
        throw a;  
    }  
    catch (float)  
    {  
        cout << "ERROR3" << endl;  
    }  
    cout << "end3" << endl;  
}
```

输出“ERROR0!”,再执行该catch子句后面的“end0”。

例子3: 异常传播的例子 (1/2)

```
#include <iostream>
int add(int a, int b)
{int res;
  try    //结果过大过小时都抛出异常
  {res = a + b;
    if (res > 128)    //抛出整型异常
      throw res;
    if (res<0)        //抛出字符串异常
      throw "Negative result!";
  }
  catch (int e) //捕捉整型异常
  {cout<<"The result is too large : "
    <<e<<endl;
    return -1;
  }
  return res; }
```

例子3: 异常传播的例子 (2/2)



```
main()
{int a, b, result;
cout<<"Enter two integers (EOF to end) :";
while (cin>>a>>b)
{try
{result = add(a, b);
if (result >= 0)
cout<<"The result is "<<result<<endl;
}
catch (...) //捕捉传播到外层的所有异常
{cout<<"Unexpected exception."<<endl;}
cout<<endl;
cout<<"Enter a integer (EOF to end) :";
} return 0; }
```

程序运行结果

Enter two integers (EOF to end): 12 66

The result is 78

Enter a integer (EOF to end): 23 456

The result is too large : 479

Enter a integer (EOF to end): -123 34

Unexpected exception.

Enter a integer (EOF to end):

异常的捕获和处理 (1/4)

- 异常处理器以关键字 `catch` 开始
- 异常处理器能带一个参数（能捕捉的异常类型），参数名可选
- 有参数名时，可以在异常处理器内使用这个参数，该参数只是抛出的异常对象的一个副本
`catch (int e) {...}`
- 没有参数名时，异常对象不从抛出点传递到异常处理器中
`catch (int) {...}`

异常的捕获和处理 (2/4)

- 程序按顺序寻找匹配的异常处理器，抛出的异常被第一个类型符合的异常处理器捕获
- 如果内层try块后面没有找到合适的异常处理器，该异常向外传播，到外层try块后面寻找
- 没有被捕获的异常将调用 `terminate` 函数，`terminate` 函数默认调用 `abort` 函数终止程序执行

异常的捕获和处理 (3/4)

- 参数列表中只有省略号的异常处理器能捕捉所有类型的异常
 `catch (...)` {语句序列}
- 异常处理器的排列顺序，可能会影响异常处理的结果

异常的捕获和处理 (4/4)

- 满足下面条件之一时，异常被捕捉
 1. 异常处理器的参数类型和抛出的异常数据的类型完全相同
 2. 异常处理器的参数类型是抛出的异常对象的基类
 3. 异常处理器的参数是基类的指针或引用，抛出的异常数据类型是派生类的指针或引用
 4. 异常处理器的参数是void*类型的指针，抛出的异常数据类型是某一种类型的指针
 5. 异常处理器为catch (...)

例子4: 异常捕获的例子 (1/3)

```
#include <iostream>
//定义基类
class base
{public:
    void show()
    {cout<<"Base object."<<endl;}
};
//定义派生类
class derived :public base
{public:
    void show()
    {cout<<"Derived object."<<endl;}
};
```

例子4: 异常捕获的例子 (2/3)

```
main()  
{int no;  
  cout<<"Input a integer please:";  
  while(cin>>no)  
  {try  
    {if ((no % 2) == 0) //抛出基类对象  
      throw base();  
    else //抛出派生类对象  
      throw derived();  
    }  
  catch(base b)  
  {cout<<"Exception:";  
    b.show();  
  }  
}
```

例子4: 异常捕获的例子 (3/3)

```
catch (derived d)
{ cout<<"Exception: ";
  d.show();
}
cout<<endl<<"Input a integer please: ";
}
return 0;
}
```

程序运行结果

Input a integer please:1

Exception:Base object.

Input a integer please:2

Exception:Base object.

Input a integer please:3

Exception:Base object.

Input a integer please:4

Exception:Base object.

Input a integer please:

学习目的检测

- 理解异常处理的意义
- 理解C++的异常处理机制
- 掌握异常的传播机制
- 掌握异常的捕获规则
- 掌握使用异常处理机制处理错误的方法
- 知道何时需要使用异常处理机制来处理错误

Thank you !